VeriSight: Master Demonstration & Technical Defense Guide

50 High-Probability Q&As; Across the Entire Tri-Engine Architecture with Spoken English Solutions & Implementation Code Paths

Presenters / Authors: Patel Kittu & Vinod Nikhil Monterio • Mirafra Technologies • October 2026

How to Ace Your Live Demonstration: This master guide is compiled specifically for your live presentation and Q&A; defense. It covers every corner of the VeriSight platform—from high-level business ROI to deep AST logic cones, pure spec-driven verification plans, and UVM 1.2 testbench synthesis. When a judge, client, or engineering director asks a question, use the **Spoken Answer** for an immediate, natural response in simple English. If they challenge you with *'Where is this implemented in your code?'*, cite the exact **Implementation Path** provided for every question.

1. High-Level Vision, Business Value & Enterprise ROI

Q1. In simple terms, what is VeriSight and what core problem does it solve?

■ ***Delivery Mindset & Cue:*** *Keep eye contact, smile, and emphasize the transition from 'multi-week manual bottleneck' to '15-second autonomous copilot'.*

■■ **Spoken Answer:** "VeriSight is an autonomous AI copilot built specifically for digital silicon design and verification. Today, verification eats up more than 70% of the entire chip development timeline, and senior engineers spend most of their days manually scrolling through 50,000-line simulation logs and tracing waveforms. VeriSight automates that whole triage and testbench cycle: it generates complete UVM testbenches from specifications, writes 50+ test verification plans, and isolates root causes in under 15 seconds."

■ `Implementation Path: verisight/wizard.py (Launcher UI) & verisight/orchestrator.py (Pipeline)`

Q2. How is VeriSight fundamentally different from tools like Synopsys Verdi or Siemens Questa?

■ ***Delivery Mindset & Cue:*** *Use the medical X-ray analogy; non-technical executives and judges grasp it immediately.*

■■ **Spoken Answer:** "Verdi and Questa are world-class waveform viewers and simulation engines, but they require a human engineer to do 100% of the manual signal tracing and thinking. VeriSight does not replace Verdi; it acts as the intelligent autonomous brain on top of it. It reads the simulation logs, traverses the circuit AST, identifies the bug using formal decision rules, and even tests a fix in a sandbox. It's like the difference between looking at an X-ray yourself versus having an expert radiologist hand you the diagnosis instantly."

■ `Implementation Path: verisight/orchestrator.py & verisight/reporter/waveform_studio.py`

Q3. How did you arrive at the '100x faster triage' claim?

■ ***Delivery Mindset & Cue:*** *State the numbers firmly: 4 to 6 hours manual vs. 14.8 seconds autonomous.*

■■ **Spoken Answer:** "We benchmarked real-world industrial verification failures across ALUs, circular FIFOs, and AXI streaming interfaces. When a regression test fails on an SoC, an engineer takes an average of 4 to 6 hours to download the log, open waveforms, back-trace clock domains, and inspect scoreboards. VeriSight runs that entire multi-agent pipeline and pinpoints the root cause in an average of 14.8 seconds. That is a direct 100-fold reduction in triage latency."

■ `Implementation Path: verisight/benchmarks/root_cause_bench.py`

Q4. Will VeriSight replace human verification engineers?

■ ***Delivery Mindset & Cue:*** *Reassure the audience that VeriSight empowers engineers as a copilot, not a replacement.*

■■ **Spoken Answer:** "No, it elevates them. There is a global shortage of more than 30,000 verification engineers right now. Instead of having high-paid senior architects waste 60% of their day doing repetitive log reading and chasing uninitialized reset bugs, VeriSight handles the routine triage so engineers can focus on high-value system architecture, critical corner cases, and meeting tape-out deadlines."

■ `Implementation Path: verisight/wizard.py & verisight/agents/agent4_reporter.py`

Q5. What is the business ROI for a semiconductor design company?

■ ***Delivery Mindset & Cue:*** *Highlight the $15M to $30M mask re-spin insurance value.*

■■ **Spoken Answer:** "The ROI comes from two huge factors: first, saving thousands of engineering hours per project, which amounts to roughly $1.8 million annually for a typical 25-person verification team. But second, and most importantly, preventing physical silicon mask re-spins. At advanced 3-nanometer and 5-nanometer process nodes, an escaped functional bug requires a re-spin costing between $15 million and $30 million. Preventing even one re-spin pays for VeriSight hundreds of times over."

■ `Implementation Path: verisight/plan/traceability.py & verisight/agents/agent5_fix_generator.py`

2. Tri-Engine Architecture & Multi-Agent Cognitive Fabric

Q6. What is the Tri-Engine Architecture and why did you consolidate the platform into 3 core engines?

■ ***Delivery Mindset & Cue:*** *Walk them through the 3 engines clearly: Debug, Plan, and TestBench.*

■■ **Spoken Answer:** "Earlier prototypes had disconnected workflows, but real chip teams work in three distinct phases: Debugging, Planning, and Testbench authoring. We consolidated VeriSight into three dedicated engines: Engine 1 is VeriSight Debug (for root-cause analysis and auto-patching); Engine 2 is VeriSight Plan (for feature extraction, 50+ scaled tests, and SVA assertions); and Engine 3 is VeriSight TestBench (for spec-grounded UVM 1.2 suite synthesis). This gives engineers a clean, intuitive entry point through our `./start` wizard."

■ `Implementation Path: verisight/wizard.py (VeriSightLauncher class)`

Q7. Why did you use 5 specialized agents instead of a single prompt to ChatGPT or Claude?

■ ***Delivery Mindset & Cue:*** *Emphasize modularity, typed contracts, and zero cognitive overload.*

■■ **Spoken Answer:** "Silicon verification requires completely different cognitive skills at each stage. One stage is log parsing, another is formal rule evaluation, another is hardware netlist tracing, another is interactive reporting, and another is code patching. If you try to do all of that in a single giant prompt, the LLM loses context, hallucinates non-existent hardware, and produces syntax errors. By dividing the work into 5 specialized agents communicating through typed JSON schemas, each agent performs its task with mathematical precision."

■ **`Implementation Path: verisight/agents/ (agent1_knowledge.py through agent5_fix_generator.py)`**

Q8. How do the agents pass data and communicate with each other?

■ ***Delivery Mindset & Cue:*** *Mention Pydantic schemas and strongly-typed JSON data contracts.*

■■ **Spoken Answer:** "They communicate through strict, typed Pydantic data schemas. Agent 1 creates `knowledge.json` containing the elaborated AST, tokenized logs, and sliced waveforms. Agent 2 reads that and produces `summary.json`. Agent 3 adds deep circuit cone analysis into `rtl_analysis.json`. Agent 5 writes `proposed.patch`, and Agent 4 compiles the final executive HTML dashboard. Because every handshake is strongly typed, no agent can pass corrupted or hallucinated data downstream."

■ **`Implementation Path: verisight/schemas/ (knowledge_schema.py, classification.py, rtl_analysis.py, fix_schema.py)`**

Q9. What happens if an LLM API call times out or encounters network failure?

■ ***Delivery Mindset & Cue:*** *Stress resilience, offline fallback, and continuous operational uptime.*

■■ **Spoken Answer:** "VeriSight has built-in graceful fallback logic. If an external LLM API fails or times out, the system automatically falls back to our deterministic offline rule engine. It continues executing Agent 3 and Agent 4 so the engineer still gets full lint analysis, VCD waveform slicing, and a comprehensive report with whatever evidence was collected, clearly flagging that LLM reasoning was unavailable."

■ **`Implementation Path: verisight/agents/base_agent.py & verisight/orchestrator.py`**

Q10. Which AI models are supported and how do you switch between them?

■ ***Delivery Mindset & Cue:*** *Mention that cloud and local open-source models work interchangeably.*

■■ **Spoken Answer:** "VeriSight features a pluggable model adapter. Out of the box, we support Google Gemini 2.5 Flash, Anthropic Claude 3.5 Sonnet, and OpenAI GPT-4o for cloud users. For air-gapped on-premise users, we connect directly via Ollama or vLLM to open-weights models like DeepSeek-Coder-V2 and Llama-3-70B. You can switch models simply by changing an environment variable or selecting an option in our `./start` wizard."

■ **`Implementation Path: verisight/config.py & verisight/agents/base_agent.py`**

Q11. How do you prevent prompt bloat and keep API costs under 5 cents per run?

■ ***Delivery Mindset & Cue:*** *Mention the 84% token reduction achieved by AST pruning and temporal slicing.*

■■ **Spoken Answer:** "Standard simulation logs and RTL files can be millions of tokens long, which would cost tens of dollars per run. VeriSight uses compiler AST pruning with `pyslang` and targeted temporal waveform slicing with `pywellen`. We strip out 84% of irrelevant code and only pass the immediate $+/- 20$ cycle logic cone surrounding the failure. This keeps average prompt size under 12k tokens, costing between 2 to 5 cents per complete debug run."

■ **`Implementation Path: verisight/parsers/pyslang_rtl.py & verisight/parsers/vcd_parser.py`**

3. Engine 1 Deep Dive — VeriSight Debug, 9 Decision Rules & AST X-Tracer

Q12. How does the 9-Rule Decision Engine guarantee zero hallucinations?

■ ***Delivery Mindset & Cue:*** *Speak firmly on the 'evidence-bounded + deterministic rule' design.*

■■ **Spoken Answer:** "We guarantee zero hallucinations by enforcing that the AI cannot invent reasons out of thin air. Agent 2 must evaluate our 9 Formal Decision Rules against concrete evidence: exact timestamps, physical signal states from the waveform, and source AST lines. If the evidence does not strictly satisfy any rule, the system is hardcoded to output 'Unknown' with alternative hypotheses rather than guessing."

■ **`Implementation Path: verisight/agents/agent2_classifier.py (_evaluate_rules() method)`**

Q13. How does VeriSight tell the difference between an RTL bug and a Testbench bug? Give a real example.

■ ***Delivery Mindset & Cue:*** *Use the RAM 1-cycle read latency example; it proves real verification competence.*

■■ **Spoken Answer:** "This is one of VeriSight's strongest features. It cross-checks the specification against both the RTL and the testbench. In our synchronous RAM testbed, the spec required a 1-clock-cycle read latency. The RTL correctly registered the read data on the clock edge, but the UVM scoreboard was checking `rdata` combinationally in the very same cycle. VeriSight recognized that the scoreboard was failing to model the pipeline latency, correctly isolating it as a Testbench Bug with 95% confidence."

■ **`Implementation Path: verisight/agents/agent2_classifier.py & verisight/analysis/failure_analysis.py`**

Q14. What is the X-Tracer algorithm and how does it find uninitialized signals?

■ ***Delivery Mindset & Cue:*** *Two layers: pyslang static AST finds reset-less registers; WaveTrace netlist confirms runtime path.*

■■ **Spoken Answer:** "Unknown state 'X' is one of the most frustrating bugs in digital hardware. X-Tracer works in two coordinated layers: First, static source inspection using `pyslang` to elaborate the AST and flag registers assigned in clock blocks but lacking a reset branch. Second, temporal VCD slicing that locates the exact cycle T_{fail} where a signal went to 'X'. WaveTrace then walks backward through the driving netlist cone from cycle $T_{\text{fail}} - 1$ to isolate the origin flip-flop."

■ **`Implementation Path: verisight/tools/xtracer_runner.py & verisight/parsers/pyslang_rtl.py`**

Q15. Waveform dumps can be 10 to 50 Gigabytes. How do you handle them without running out of memory?

■ ***Delivery Mindset & Cue:*** *Explain Temporal Waveform Slicing: only +/- 20 cycles around failure.*

■■ **Spoken Answer:** "We never load an entire 50GB waveform into memory or pass raw VCD to an LLM. We built a high-speed Rust-based parser using `pywellen` and `vcdvcd` that performs 'Temporal Waveform Slicing'. We extract only the 20 clock cycles immediately surrounding the assertion failure timestamp, and only for the signals directly connected to the active logic cone. That compresses gigabytes of trace data into a clean, sub-megabyte JSON slice."

■ **`Implementation Path: verisight/parsers/vcd_parser.py`**

Q16. How does VeriSight detect Clock Domain Crossing (CDC) and metastability bugs?

■ ***Delivery Mindset & Cue:*** *Demonstrate knowledge of multi-flop synchronizers and async clock domains.*

■■ **Spoken Answer:** "Inside Agent 3, our dedicated CDC engine inspects all signals crossing between asynchronous clock domains. It checks the AST to verify whether multi-flop synchronizers, handshake synchronizers, or asynchronous FIFOs are present. If data is sampled across asynchronous domains without adequate synchronization, it immediately flags the crossing with source and destination clock domain tags."

■ **`Implementation Path: verisight/agents/agent3_rtl_analyzer.py & verisight/analysis/extended_checks.py`**

Q17. How does VeriSight detect FSM deadlocks and illegal state transitions?

■ ***Delivery Mindset & Cue:*** *Explain that FSM state graphs are statically constructed from AST case statements.*

■■ **Spoken Answer:** "Agent 3 constructs a state transition graph directly from the SystemVerilog `case` or `if-else` state decoding logic. It checks for unreachable states, dead-end states with no exit transition, and default branch omissions. If a simulation hangs, it correlates the current state register in the waveform against the graph to prove the FSM is trapped in an infinite deadlock."

■ **`Implementation Path: verisight/agents/agent3_rtl_analyzer.py (_check_fsm_deadlocks())`**

Q18. What if the simulation produces no waveform dump and only text logs?

■ ***Delivery Mindset & Cue:*** *Stress that the tool works with or without waveforms.*

■■ **Spoken Answer:** "VeriSight is completely modular. If a VCD or FSDB waveform dump is provided, X-Tracer and waveform studio are activated. If only a simulation log and RTL/TB are provided, VeriSight continues seamlessly using log tokenization, AST inspection, and spec cross-referencing. It will simply inform the user that waveform evidence was not present."

■ **`Implementation Path: verisight/parsers/log_parser.py & verisight/agents/agent1_knowledge.py`**

Q19. Does X-Tracer handle active-low vs. active-high and asynchronous vs. synchronous resets?

■ ***Delivery Mindset & Cue:*** *Demonstrate knowledge of synthesizable SV reset patterns.*

■■ **Spoken Answer:** "Yes. `pyslang_rtl.py` inspects the AST sensitivity list and conditional branches inside `always_ff` and `always` blocks. It checks whether `negedge` or `posedge` reset triggers are correctly matched with active-low (`!rst_n`) or active-high (`rst`) conditions, flagging missing reset clauses or inverted reset polarity immediately."

■ **Implementation Path: `verisight/parsers/pyslang_rtl.py`**

4. Engine 2 Deep Dive — VeriSight Plan, 50+ Testcases & Dynamic Coverage Closure

Q20. What is VeriSight Plan and how does it synthesize verification plans from natural language specs?

■ ***Delivery Mindset & Cue:*** *Emphasize that it translates English specifications into executable verification plans.*

■■ **Spoken Answer:** "VeriSight Plan (Engine 2) translates architectural specification documents—such as PDFs, Word files, or Markdown datasheets—into complete, executable verification plans. It decomposes the spec into atomic requirements, synthesizes 50+ scaled testcases across 8 categories, generates multi-covergroup models with 90+ coverage points, and outputs formal SystemVerilog assertions."

■ **Implementation Path: `verisight/plan/planner_orchestrator.py` & `verisight/plan/feature_extractor.py`**

Q21. What happens if I don't provide an RTL file? Does it hallucinate or fall back to a default ALU?

■ ***Delivery Mindset & Cue:*** *Highlight the 'shift-left' capability: planning test suites before RTL is written.*

■■ **Spoken Answer:** "No, we specifically engineered VeriSight Plan to support pure spec-driven mode where RTL is completely optional. If no RTL file is provided, it does not fall back to ALU or hallucinate code; it performs pure architectural planning directly from the specification document (such as `i2c_design_spec.pdf`). This allows verification teams to shift left and build complete test plans before RTL design even starts."

■ **Implementation Path: `verisight/wizard.py (handle_plan())` & `verisight/plan/planner_orchestrator.py`**

Q22. How does VeriSight generate 50+ unique testcases instead of just 10 generic ones?

■ ***Delivery Mindset & Cue:*** *List the 8 categories: Directed, Random, Reset, Corner, Error, Performance, Concurrency, Protocol.*

■■ **Spoken Answer:** "Our `test_planner.py` uses an 8-category decomposition model. Instead of asking an LLM for 'some testcases', it systematically prompts across eight rigorous verification dimensions: Directed tests, Constrained-Random sequences, Reset/Initialization checks, Boundary/Corner cases, Error/Negative scenarios, Performance/Backpressure tests, Concurrency/Race cases, and Protocol-Specific edge conditions. This guarantees at least 50 unique, highly targeted test scenarios."

■ **Implementation Path: `verisight/plan/test_planner.py`**

Q23. How does the Coverage Architect synthesize 90+ coverage points and 150+ bins?

■ ***Delivery Mindset & Cue:*** *Mention cross-coverage, state transitions, and illegal state bins.*

■■ **Spoken Answer:** "In `coverage_architect.py`, the engine generates multi-covergroups covering state machines, data values, and protocol sequences. It defines individual bins for legal values, illegal state bins, cross-coverage between control and data signals, and transition bins (e.g. `IDLE => READ => WAIT => IDLE`). This produces 90+ coverage points and over 150 bins ready for inclusion in a UVM subscriber."

■ `Implementation Path: verisight/plan/coverage_architect.py`

Q24. How does VeriSight synthesize SystemVerilog Assertions (SVA) from text specs?

■ ***Delivery Mindset & Cue:*** *Mention formal implication operators |-> and |=> with reset guards.*

■■ **Spoken Answer:** "`assertion_architect.py` reads natural language protocol requirements and translates them into formal concurrent SVA properties. It generates correct overlapping (`|->`) and non-overlapping (`|=>`) implication operators, clocking event bindings, and `disable iff (!rst_n)` reset guards, producing 30+ production-grade SVA properties."

■ `Implementation Path: verisight/plan/assertion_architect.py`

Q25. How does VeriSight ingest live simulator coverage databases (Questa UCDB / Synopsys vDB)?

■ ***Delivery Mindset & Cue:*** *Explain the live comparison between planned features vs. actual simulator hit bins.*

■■ **Spoken Answer:** "Our `coverage_ingest.py` and `coverage_correlator.py` modules parse Questa UCDB XML/text exports and Synopsys vDB reports. It maps actual hit counts from the simulator directly against the planned covergroups, calculating true feature completion percentages in real time."

■ `Implementation Path: verisight/plan/coverage_ingest.py & verisight/plan/coverage_correlator.py`

Q26. How does VeriSight help teams close the last 15% stubborn coverage holes?

■ ***Delivery Mindset & Cue:*** *Stress that AI directed stimulus turns 6-8 weeks of random seeds into 4 days.*

■■ **Spoken Answer:** "When teams get stuck at 85% coverage, brute-force random testing produces diminishing returns. VeriSight's Coverage Correlator identifies the exact zero-hit bins in the UCDB report—like multi-beat backpressure during FIFO full—and automatically synthesizes directed UVM sequence constraints crafted specifically to hit those unreached states in just days."

■ `Implementation Path: verisight/plan/coverage_correlator.py`

Q27. How does VeriSight satisfy automotive safety standards like ISO 26262 ASIL-D and DO-254?

■ ***Delivery Mindset & Cue:*** *Mention ISO 26262 ASIL-D, DO-254, and the bidirectional traceability matrix.*

■■ **Spoken Answer:** "In `traceability.py`, VeriSight generates a complete bidirectional traceability matrix exported to CSV and HTML (`traceability_matrix.csv`). It links every functional safety requirement directly to its corresponding test case, SVA assertion, and coverage point, giving automotive auditors an indisputable sign-off trail."

■ `Implementation Path: verisight/plan/traceability.py`

5. Engine 3 Deep Dive — Spec-Driven UVM 1.2 Testbench Synthesis

Q28. What is the Spec-Driven UVM Testbench Synthesis engine and what is the 'Spec-as-Truth' principle?

■ ***Delivery Mindset & Cue:*** *Speak passionately about 'Spec-as-Truth'—it is the single biggest technical differentiator.*

■■ **Spoken Answer:** "Engine 3 (`spec_driven_tb_agent.py`) synthesizes complete, cycle-accurate UVM 1.2 verification suites from specifications. The 'Spec-as-Truth' principle is our fundamental rule: traditional AI tools read RTL to write testbenches, which means they copy the designer's bugs into the reference model! VeriSight treats the Architectural Specification as the sole functional truth. Its reference models and scoreboards independently discover RTL defects rather than reproducing them."

■ **`Implementation Path: verisight/agents/spec_driven_tb_agent.py & verisight/uvm/uvm_architect.py`**

Q29. Why shouldn't an AI generate testbenches by reading the RTL code?

■ ***Delivery Mindset & Cue:*** *Use the simple arithmetic example (data+1 vs data+2) to make the danger crystal clear.*

■■ **Spoken Answer:** "If the RTL designer made a mistake—like calculating `data + 1` instead of `data + 2`—an AI that reads the RTL will write a reference model that expects `data + 1`. The simulation will pass with 100% false confidence, and the bug will escape into silicon! By grounding the reference model strictly in the English specification, VeriSight ensures the testbench catches the RTL bug every single time."

■ **`Implementation Path: verisight/agents/spec_driven_tb_agent.py`**

Q30. What are the 14 VIP components that VeriSight generates?

■ ***Delivery Mindset & Cue:*** *Recite the standard UVM components to prove complete VIP coverage.*

■■ **Spoken Answer:** "VeriSight generates a full, production-grade UVM 1.2 environment comprising 14 modular files: `_if.sv` (interface), `_seq_item.sv` (transaction), `_sequencer.sv`, `_driver.sv`, `_monitor.sv`, `_agent.sv`, `_reference_model.sv` (golden predictor), `_scoreboard.sv`, `_coverage.sv` (subscriber), `_sequence.sv`, `_env.sv`, `_test.sv`, `_pkg.sv` (package), and `_tb_top.sv` (top testbench)."

■ **`Implementation Path: verisight/uvm/uvm_architect.py & verisight/uvm/matrix_synthesizer.py`**

Q31. How does the behavioral Golden Reference Model work inside the UVM environment?

■ ***Delivery Mindset & Cue:*** *Explain the analysis FIFO connection between monitor, reference model, and scoreboard.*

■■ **Spoken Answer:** "The reference model (`_reference_model.sv`) is a transactional predictor connected between the monitor and the scoreboard via UVM analysis FIFOs. It models the ideal behavioral state transitions and arithmetic calculations specified in the datasheet. When a transaction completes, the scoreboard compares the DUT's actual output against the reference model's predicted output."

■ **`Implementation Path: verisight/uvm/uvm_architect.py (_generate_reference_model())`**

Q32. How do you guarantee the generated SystemVerilog UVM code compiles without syntax errors?

■ ***Delivery Mindset & Cue:*** *Mention that pyslang compiles and validates all SV code before saving.*

■■ **Spoken Answer:** "In `sv_validate.py`, every generated file is validated against the official IEEE-1800 SystemVerilog grammar using the `pyslang` parser. If a missing semicolon, undeclared variable, or improper macro usage is detected, VeriSight immediately catches it and fixes it before presenting the files to the user."

■ **`Implementation Path: verisight/tools/sv_validate.py`**

Q33. How does the 10-Stage Reasoning Pipeline execute during testbench generation?

■ ***Delivery Mindset & Cue:*** *Walk through the 10 stages smoothly as an end-to-end synthesis pipeline.*

■■ **Spoken Answer:** "`spec_driven_tb_agent.py` executes a structured 10-stage sequence: (1) Spec ingestion -> (2) Requirement grounding -> (3) Protocol state extraction -> (4) Interface signal mapping -> (5) Sequence item definition -> (6) Driver & Monitor creation -> (7) Behavioral Golden Reference Model -> (8) Scoreboard comparator -> (9) Coverage subscriber -> (10) Top module integration & syntax sign-off."

■ **`Implementation Path: verisight/agents/spec_driven_tb_agent.py (run_spec_driven_pipeline())`**

6. Automated Repair, Sandbox Safety & Regression Sign-Off

Q34. How does Agent 5 generate code patches without breaking existing functionality?

■ ***Delivery Mindset & Cue:*** *Emphasize minimal git diffs and sandbox verification.*

■■ **Spoken Answer:** "Agent 5 operates under a strict 'minimal-code-region' policy. It modifies only the specific lines directly responsible for the defect, keeping changes as small as possible. Most importantly, it creates a clean git patch (`proposed.patch`) and immediately compiles it in a temporary sandbox. If the patched code fails syntax checking or fails the regression, the fix is declined with full reasoning."

■ **`Implementation Path: verisight/agents/agent5_fix_generator.py`**

Q35. What is the Isolated Compile Sandbox and how does it prevent broken code from being committed?

■ ***Delivery Mindset & Cue:*** *Stress that the main repository is never touched until sandbox verification passes.*

■■ **Spoken Answer:** "In `sim_runner.py` and `eda_runner.py`, VeriSight copies the design files into an isolated `/tmp` workspace. It applies the patch in that sandbox and executes the simulator (e.g. Verilator or VCS). If the compiler throws an error, or if existing regression tests fail, the patch is rejected immediately. Your main working directory is never touched."

■ **`Implementation Path: verisight/tools/sim_runner.py & verisight/tools/eda_runner.py`**

Q36. Is VeriSight an autopilot that automatically overrides human code, or a copilot?

■ ***Delivery Mindset & Cue:*** *Reinforce the core philosophy: 'Copilot, not autopilot; human in the loop'.*

■■ **Spoken Answer:** "VeriSight is strictly a copilot. The engineer always retains 100% final approval. The generated fix is saved as a standard unified `.patch` file alongside an interactive HTML diff viewer in the dashboard. The engineer can review the patch, adjust it, or discard it with a single click."

■ **`Implementation Path: verisight/agents/agent4_reporter.py & verisight/agents/agent5_fix_generator.py`**

Q37. What happens if the proposed patch fails the regression test suite?

■ ***Delivery Mindset & Cue:*** *Show that failure rejection is transparent and documented in the report.*

■■ **Spoken Answer:** "If the patched code fails regression in the sandbox, Agent 5 logs the compile error or failing assertion, decrements its confidence score, and outputs an explanation in `report.html` explaining why the candidate fix was rejected, along with alternative debugging recommendations for the human engineer."

■ **`Implementation Path: verisight/agents/agent5_fix_generator.py`**

7. Security, Sovereign Air-Gap & Semiconductor IP Confidentiality

Q38. Chip RTL is proprietary and classified. How do you guarantee no IP leakage to the cloud?

■ ***Delivery Mindset & Cue:*** *Speak with total conviction—IP security is the #1 requirement for semiconductor firms.*

■■ **Spoken Answer:** "Data privacy is our number one architectural priority. VeriSight provides two distinct enterprise security layers: First, **Sovereign Air-Gapped Mode**, where the entire platform runs on your on-premise compute cluster with zero internet access, using local models like DeepSeek-Coder. Second, if you do choose to use a cloud API, our built-in **RTL Anonymizer** automatically strips proprietary module names, signal identifiers, and comments before sending prompts."

■ **`Implementation Path: verisight/utils/airgap.py & verisight/utils/anonymizer.py`**

Q39. Can VeriSight run 100% offline in a completely disconnected air-gapped laboratory?

■ ***Delivery Mindset & Cue:*** *Clear, unambiguous: Yes, 100% offline verified.*

■■ **Spoken Answer:** "Yes, 100%. We have tested and verified VeriSight running in completely disconnected environments with network adapters disabled. All AST parsing, VCD slicing, rule engines, local vector search, and local LLM inference via Ollama or vLLM run entirely on localhost."

■ **`Implementation Path: verisight/utils/airgap.py`**

Q40. Does VeriSight or its creators train AI models on customer RTL code?

■ ***Delivery Mindset & Cue:*** *Reassure customer confidentiality and zero-data-retention compliance.*

■■ **Spoken Answer:** "Never. VeriSight does zero model training and does not store customer RTL. In air-gapped mode, there is physically no outbound connection. When using commercial cloud APIs, we adhere to enterprise zero-data-retention API policies where customer inputs are never used for model training."

■ **`Implementation Path: verisight/config.py`**

Q41. What physical hardware is needed to deploy VeriSight on-premise?

■ ***Delivery Mindset & Cue:*** *Give concrete specs: standard PC for core, 1 GPU for local LLM.*

■■ **Spoken Answer:** "For the deterministic rule engines, AST parsing, and reporting, any standard Linux x86 workstation or VM with 4 CPU cores and 8GB RAM is plenty. If you wish to run a local 16-billion or 33-billion parameter model like DeepSeek-Coder locally, an enterprise workstation with a single NVIDIA RTX 4090 or A100 GPU running Ollama or vLLM delivers instant, local inference."

■ **`Implementation Path: verisight/config.py`**

8. EDA Simulator Ecosystem, CI/CD & Platform Utilities

Q42. How does VeriSight integrate into existing Makefiles and simulation scripts?

■ ***Delivery Mindset & Cue:*** *Explain both Passive mode (just read logs) and Active mode (run sims).*

■■ **Spoken Answer:** "VeriSight requires zero changes to your existing testbenches or Makefiles. It can operate in non-intrusive 'passive mode' by simply reading the simulation logs and VCD dumps your current regression produces. Or it can run in 'active mode' where our Closed-Loop Runner wraps simulator invocation directly."

■ `Implementation Path: verisight/tools/eda_runner.py`

Q43. Which commercial and open-source simulators are supported out of the box?

■ ***Delivery Mindset & Cue:*** *List VCS, Questa, Xcelium, Verilator, and Icarus Verilog.*

■■ **Spoken Answer:** "All major Big-3 commercial engines: Synopsys VCS, Siemens QuestaSim/ModelSim, and Cadence Xcelium. It also fully supports open-source Verilator and Icarus Verilog. Our log parser has pre-trained tokenizers for VCS UVM error strings, Questa assertion reports, and standard `$display` outputs."

■ `Implementation Path: verisight/tools/sim_runner.py`

Q44. How does VeriSight run in automated nightly CI/CD regression pipelines?

■ ***Delivery Mindset & Cue:*** *Describe the automated CI workflow ending in Jira/Slack.*

■■ **Spoken Answer:** "VeriSight features a complete headless CLI interface. You can add a single line to your Jenkins, GitLab CI, or GitHub Actions pipeline. If a test exits with non-zero status, VeriSight automatically wakes up, executes triage, generates `error.json` and `report.html`, and can post the diagnostic summary directly to a Slack channel or Jira bug ticket."

■ `Implementation Path: verisight/wizard.py & verisight/orchestrator.py`

Q45. What is the Live Web Cockpit and how do the synchronized waveform cursors work?

■ ***Delivery Mindset & Cue:*** *Describe clicking an error and watching the waveform cursor jump to 450ns.*

■■ **Spoken Answer:** "Running `./start serve` launches our FastAPI web studio. In `waveform_studio.py`, the frontend correlates simulation log error lines with exact physical clock edges. When an engineer clicks an error in the triage window, the waveform canvas automatically jumps the inspection cursor to the exact assertion timestamp (e.g. 450ns), with full bus radix decoding."

■ `Implementation Path: verisight/server/app.py & verisight/reporter/waveform_studio.py`

Q46. What is the Cloud EDA Playground Exporter?

■ ***Delivery Mindset & Cue:*** *Mention 1-click cloud sharing to EDA Playground without local tools.*

■■ **Spoken Answer:** "Running `./start edaplayground` packages the synthesized UVM testbench, RTL files, and run scripts into a single clean bundle and automatically generates a shareable web link to edaplayground.com. This allows verification leads to share working simulations with remote team members or clients in one click."

■ `Implementation Path: verisight/tools/eda_playground_exporter.py`

9. Tough Live Demo Gotchas & Curveball Questions

Q47. What if the specification itself is wrong, incomplete, or contradicts itself?

■ ***Delivery Mindset & Cue:*** *Highlight that resolving spec ambiguities prevents team friction.*

■■ **Spoken Answer:** "That is why Rule 9 and our 'Spec Bug' classification exist. If the specification contains contradictory requirements—for instance, one section specifies active-low reset while another requires active-high—VeriSight flags it as a 'Spec Bug' with exact clause references, preventing engineers from wasting days debating whether the bug belongs to RTL or Verification."

■ `Implementation Path: verisight/agents/agent2_classifier.py (Spec Bug rule)`

Q48. What if the LLM produces invalid SystemVerilog syntax or hallucinations during the demo?

■ ***Delivery Mindset & Cue:*** *Explain the compiler self-correction retry loop and template fallback.*

■■ **Spoken Answer:** "We don't allow raw LLM output to reach the user. In `sv_validate.py`, all generated code is pre-compiled through `pyslang`. If any syntax error is detected, an automatic self-correction loop feeds the compiler diagnostic back to the LLM to fix the syntax before display. If it fails twice, the system falls back to our deterministic template library."

■ `Implementation Path: verisight/tools/sv_validate.py & verisight/agents/base_agent.py`

Q49. What is the false-positive rate of VeriSight and how was it measured?

■ ***Delivery Mindset & Cue:*** *Stress the <3% rate and the multi-stage consensus filter.*

■■ **Spoken Answer:** "In our regression benchmark suites, the false-positive rate is under 3%. This is because of our multi-stage consensus filter: a root cause is only accepted if the log error signature, the AST code structure, and the 9-rule formal engine all agree. If there is any contradiction, the engine deliberately outputs 'Unknown' with alternative hypotheses rather than giving a false lead."

■ `Implementation Path: verisight/benchmarks/root_cause_bench.py`

Q50. If you had to summarize VeriSight in one compelling closing sentence for executives, what would it be?

■ ***Delivery Mindset & Cue:*** *Deliver with passion, conviction, and a confident closing smile.*

■■ **Spoken Answer:** "VeriSight transforms silicon verification from a slow, painful multi-week debugging bottleneck into an automated, 15-second closed-loop copilot—empowering chip teams to tape out faster, with lower costs, and zero hallucinations."

■ `Implementation Path: verisight/wizard.py`